MobileMoE

What should a Mixture of Experts look like if it has to run on a phone?

Alexander Schneier

Chen, Huang, Chang, Szwejbka, Desai, Liu, Chandra, Krishnamoorthi

Meta AI · arXiv 2605.27358 · May 2026

THE QUESTION

The cloud went sparse. The phone stayed dense.

FRONTIER MODELS

Mixture of Experts

DeepSeek-V3 · Llama 4 · Qwen3 · gpt-oss

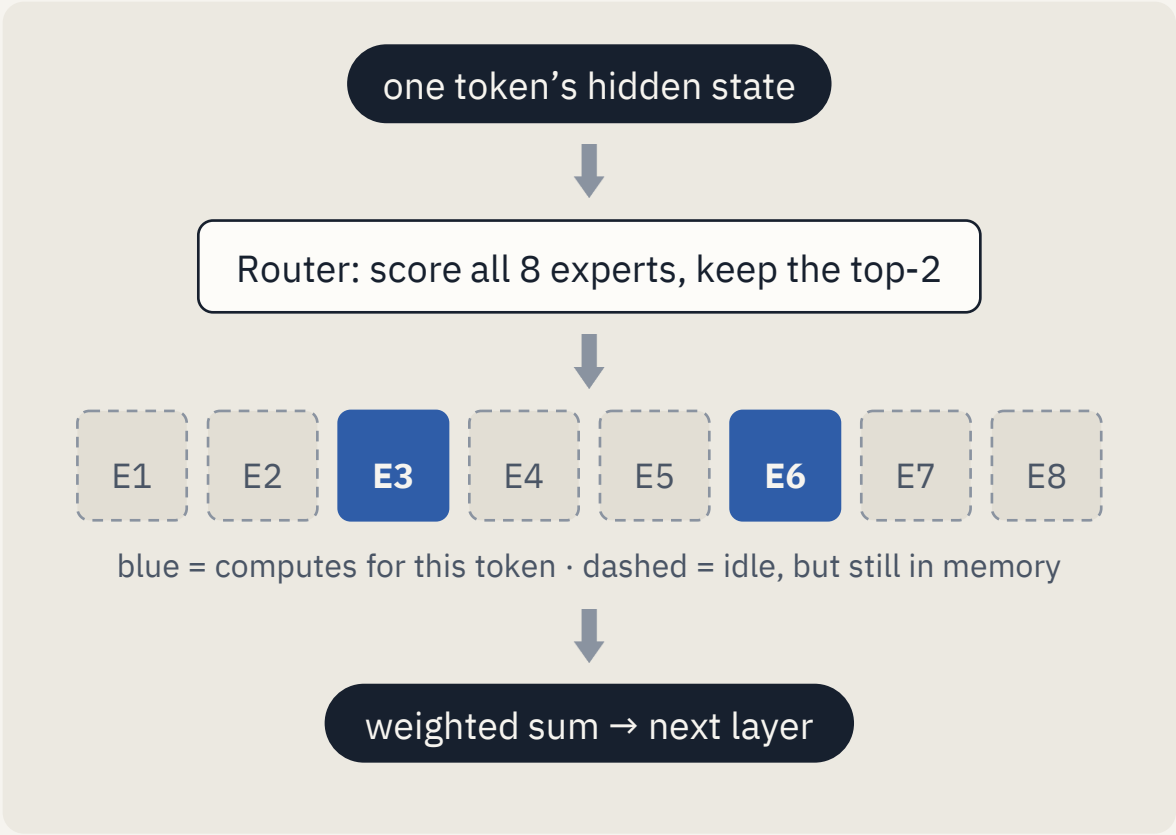
ON-DEVICE MODELS

Dense

MobileLLM · Gemma 3 · SmolLM2 · Llama 3.2 1B

What is preventing mobile models from being Mixture of Experts?

Mixture of Experts in two minutes



Each Transformer feed-forward block becomes E smaller FFNs (“experts”) plus a learned router that picks k of them per token.

COMPUTE PER TOKEN

scales with k active experts

MEMORY

scales with all E experts

DeepSeek-V3: 671B parameters stored, ~37B used per token.

2 · THE CONSTRAINT

On a phone, memory is the budget, not FLOPs

Server

Memory is shardable across GPUs, and many users are batched together.

→ optimize training and inference FLOPs

Phone

One app gets roughly 5 GB of DRAM. Batch size 1: every generated token reread the active weights.

→ weight memory and bandwidth dominate

WHAT THE PAPER OPTIMIZES

Minimize loss subject to training FLOPs, inference FLOPs (set by active params), **and** memory:

$$M = \tfrac{1}{2} \cdot N_{total}\ (\text{INT4 weights, GB}) + \text{INT8 KV cache} \leq 5\ \text{GB}$$

3 · THE CORE RESULT

An on-device MoE scaling law

Loss(N_act, D, E, x)

N_act	active params → inference FLOPs
D	training tokens
E	number of experts → total params → **memory**
x	granularity g, shared expert s: change the architecture, not the param counts

Fix E and it reduces to Chinchilla; fix x and it reduces to the routed-MoE law of Clark et al. (2022).

The sweep

Active: 0.3B, 0.5B, 0.9B

Experts E: 1, 2, 4, 8, 16, 32

Granularity g: 1, 2, 4, 8, 16

Shared expert: with / without

Data: 100–500B tokens per run

At fixed N_act the knobs decouple: only E moves memory; g and s keep both N_act and N_total fixed. So: pick E for the memory budget, then tune g and s for free.

3 · THE CORE RESULT

Three knobs, three findings

KNOB 1 · HOW SPARSE?

E = 8

MoE beats dense at any memory budget above 0.25 GB. Past 8 experts, gains flatten while memory keeps growing.

KNOB 2 · HOW FINE?

g = 8

Split each expert into 8 slices: 64 small experts, top-8. Same memory, same FLOPs, lower loss.

KNOB 3 · SHARED EXPERT?

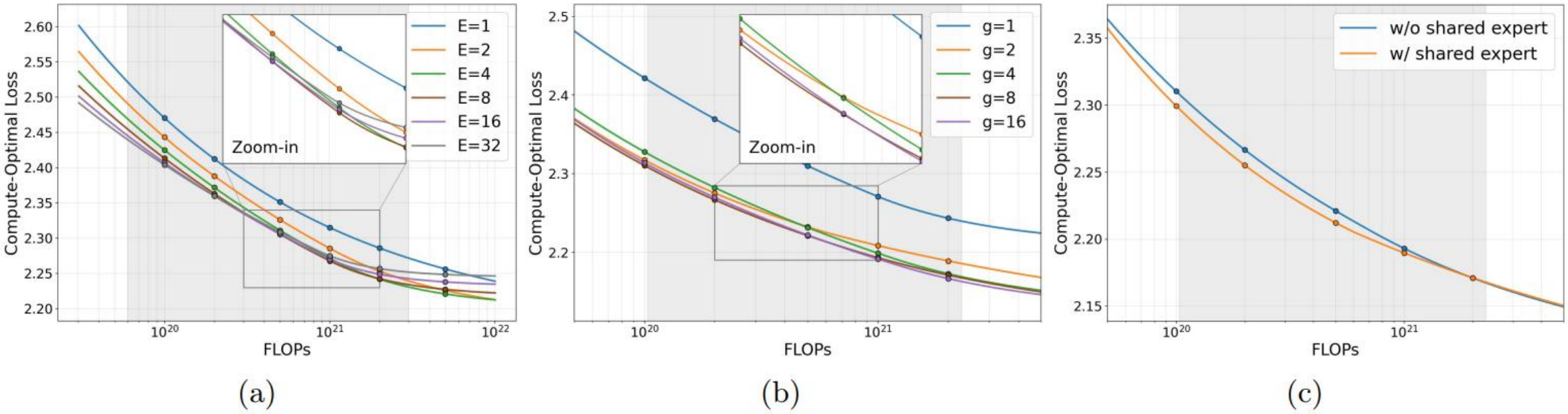
Yes

Swap 4 of the 8 active slices for one always-on expert of the same size. Same params, lower loss.

MobileMoE layer: 60 routed experts + 1 shared expert, top-4 routing

Moderate sparsity + fine-grained + shared: memory-optimal and compute-optimal at the same time.

The evidence: loss vs. compute for each knob



(a) More experts, lower curve, with little gap from 8 to 32.

(b) Every g > 1 sits below g = 1; g = 8 ≈ g = 16.

(c) Shared expert is below routed-only at every budget.

The models, and the dense baseline they race

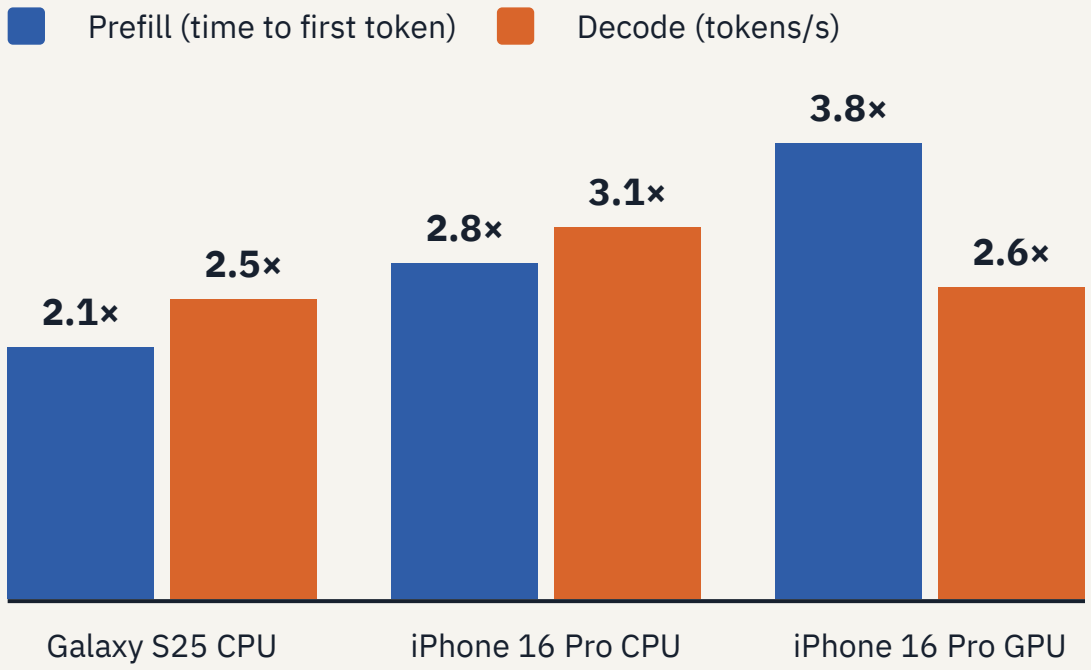
Model (INT4)	Active	Total	Weights	Avg acc.
MobileLLM-Pro (dense)	1.1B	1.1B	0.55 GB	45.5
MobileMoE-S	0.27B	1.3B	0.68 GB	44.0
MobileMoE-M	0.53B	2.8B	1.48 GB	52.5
MobileMoE-L	0.92B	5.3B	2.75 GB	57.8

The fair fight is the first two rows: similar weight memory and accuracy, but MobileMoE-S has ~4× fewer active parameters.

Phones: Samsung Galaxy S25 (Snapdragon 8 Elite) and iPhone 16 Pro (A18 Pro), ExecuTorch, plus a custom fused MoE kernel since no mobile stack had one. Training, in one line: ~6T-token pre-training → mid-training → SFT → INT4 quantization-aware training, all open data.

4 · RESULTS ON REAL PHONES

Same memory, 2–4× faster on the phone



Speedup of MobileMoE-S over dense MobileLLM-Pro, 1k-token prompt

1.8–3.8×

faster prefill

2.2–3.4×

faster decode

−22%

peak RAM at 8k context (1.49 vs. 1.91 GB)

Why: prefill is compute-bound, decode is bandwidth-bound, and both scale with **active** params. Galaxy S25 at 1k: 112 vs. 46 tokens/s.

4 · RESULTS ON REAL PHONES

And the accuracy holds up

2–4×

fewer inference FLOPs than leading dense on-device LLMs

MobileMoE-M (0.53B active) beats Qwen3.5-2B (1.9B active): 55.3 vs. 50.8

~ 60%

fewer parameters than OLMoE-1B-7B at matched accuracy

MobileMoE-M: 0.5B / 2.8B vs. 1.3B / 6.9B (active / total); 55.4 vs. 52.4

~ 5×

less memory: 4-bit MobileMoE-L beats full-precision OLMoE

2.75 GB vs. ~13.8 GB; 57.8 vs. 55.6

Not a clean sweep: Qwen3.5-2B (2× MobileMoE-L's active params) still leads on reasoning and instruction following.

5 · TAKEAWAY

Dense is not the right default on a phone anymore

What it settles

A well-shaped MoE is memory- *and* compute-optimal versus dense under a phone budget.

The shape: moderate sparsity, fine-grained experts, one shared expert.

The gain survives 4-bit quantization and shows up as real latency.

What it leaves open

It assumes **every expert fits in RAM** (≤ 5 GB; MobileMoE-L peaks at 4.71 GB).

Real prompts touch more experts: peak RAM is 1.2–2.1× higher than with dummy prompts.

Questions?